MANCHESTER 1824
The University of Manchester

Department of Electrical and Electronic Engineering

Embedded Systems Project 2023-24

DESIGN REPORT #2

Autonomous Line-Following Buggy: Software, Controls, Sensors, Circuits and Chassis Report

Group Number: 4

Group members:	ID Number	I confirm that this is the group's own work.
Benjamin McKay	11077761	Yes
Zhenhao Yu	11054962	Yes
Ga Eun Kim	11053128	Yes
Georgia Champion	11087390	Yes
Jingyi Li	11477942	Yes

Tutor: Emad Alsusa

Date: 30/11/2023

Contents

1 Introduction 2
2 Software 3
2.1 Sensor operation and constraints 3
2.2 Context diagrams 4
2.3 Use case diagrams 5
2.4 Object specification - function prototypes 6
3 Line Sensor Characterisation 6
3.1 Section introduction 6
3.2 Comparing relevant line sensors 7
3.3 Pairing emitter and detector 7
3.4 Data analysis 7
3.5 Line sensor selection 10
4 Circuit Diagram of Line Sensors 10
4.1 Introduction 10
5 Non-Line Sensors 11
5.1 Encoders 11
5.2 Battery monitor 11
5.3 Motor current 12
5.4 Implementation 12
5.5 Alternative sensors 12
6 Control System 13
6.1 Detecting the line 13
6.3 PID controllers and bang-bang controllers 14
6.4 Control objects and applicable control algorithms 16
6.5 Proportional Control 16
6.6 Interrelationship between control algorithms and sensor design 16
6.7 Impact of relationship between algorithms and control objects on the sensor setup, circuit design, and interface with the microcontroller 17
6.8 Sensor implementation 17
6.9 Control algorithm implementation 18
6.10 Line breaks and sunlight 18
7 Hardware 18
7.1 Introduction 18
7.2 Selection of best design 18
7.3 Stress analysis of chassis 19
7.4 Dimensioned 2D drawings and design considerations 20
7.5 Material selection 20
8 Conclusion 21
9 References 22

1 Introduction

This design report is integral to the successful realisation of the project's primary objectives - constructing an autonomous buggy capable of precisely following an unknown track's line, even through curves, and without deviating from the white line. There are a number of crucial steps within this report that first must be undertaken to achieve this goal. The first key step is that there must be a system in place that allows the vehicle to detect the line, and be able to alter its speed and course in response to changes in the line. This important step requires the integration of software, physical hardware and a detailed control system that tracks the line. These three key parts of the buggy are considered in section 2, 3 and 6 respectively. The reason that these sections are so important is they outline the plan of how the microcontroller will be programmed to respond to the changing events and lines on the course.
Moreover, not only does this report tackle the process of how the line will be followed, it also addresses the key issues of hardware related questions, mainly the Printed Circuit Board (PCB) layout and the chassis design. These are two essential areas that the project is reliant on as the PCB is needed to allow the electronic line sensors to be able to function, and the chassis is a necessary platform to transport all the components across the track. However, when designing both the PCB and the chassis, it is essential to remember the primary aim of the project, for a buggy to navigate a track, and in order to fulfil this aim it is important that the physical size of both the chassis and PCB are not so large that the buggy cannot fit the track (at its narrowest 280 mm), and yet not so small that the respective parts are unable to fit. Additionally, this report will also consider some other ways to gain advantages over the competition for the race day event around the track. As such, not only is material choice carefully considered, a number of optional extra features and hardware options are considered for their worth.

2 Software

2.1 Sensor operation and constraints

When the buggy is turned on, its line sensors detect a white line on the ground. The system follows this line, using the line sensors and speed sensors to calculate the direction and velocity required. The motors on each wheel change velocity based on this calculation, which it receives as a signal. Other features include a battery monitor that uses a current sensor to send information on battery usage to the system. Additionally, the encoders monitor speed and slow the buggy down if it reads a velocity that is too high (for example, going down the 18 degree slope). The buggy stops when it reaches the end of the line. When it receives a signal through its Bluetooth Low Energy (BLE) module, it turns around and returns to the starting point.

Program memory is 512 kB and data memory is 96 kB, which are both enough for the buggy to function as needed, as long as the code is written efficiently. For smooth operation of the buggy, the response times should be quick so it has time to react to changes in the line, but if they are too fast it would increase power for little benefit.

Assuming a maximum speed of 1.8 m/s, the buggy would move 1 mm every 0.001/1.8 = 0.556 ms. For the sensors to send readings every 5 mm, the required time between samples would be 2.78 ms. Rounding down to 2.5 ms gives the rate at which voltage is measured at the sensors at 400 Hz. As the microcontroller should

sample multiple times during each bit to ensure it is as close to the centre of the bit as possible, the sampling rate for the software is 400 * 8 = 3200 Hz. For the buggy to move consistently, it should process the samples and adjust the motors' speeds within two sensor voltage readings, which is 5 ms.

2.2 Context diagrams

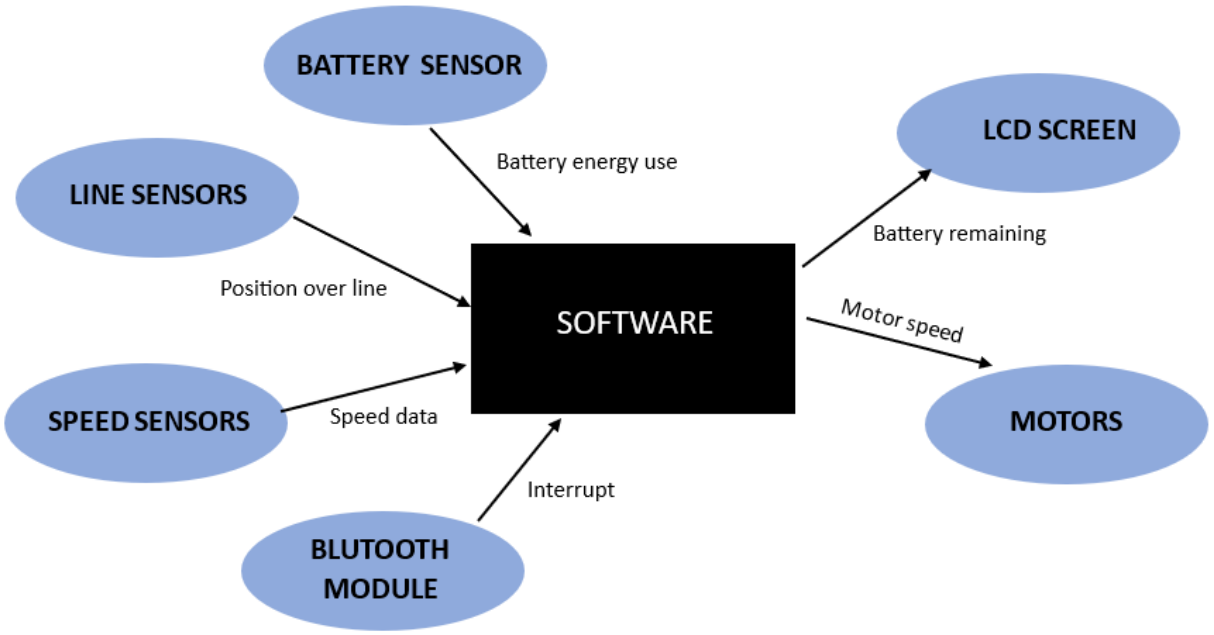


Figure 1 - Context diagram

Message	Arrival pattern	Data content	Size	Pin interaction with microcontroller
Speed data	Periodic	Flash/pulse per rotation	32 bits	TIM2,TIM5 have quadrature encoder interface mode
Position over line	Periodic	Voltage reading of sensor	12 bits (ADC)	5 analogue in pins
Motor speed	Continuous	Average voltage of PWM	N/A, continuous voltage	Timer peripherals (TIM); Two GPIO pins (each) to motor drive board to control mode and direction
Battery energy use	Periodic	Battery capacity estimation in milliamp hours and percentage left	1 bit (serial communication) – 1-wire output speed 16kbps	One GPIO pin
Bluetooth module	Aperiodic	Trigger interrupt routine	up to 251 byte data payload	UART serial port
Battery remaining	Periodic	Percentage of battery remaining	32 bits (float)	None - Display on microcontroller

Table 1 - Table of messages

2.3 Use case diagrams

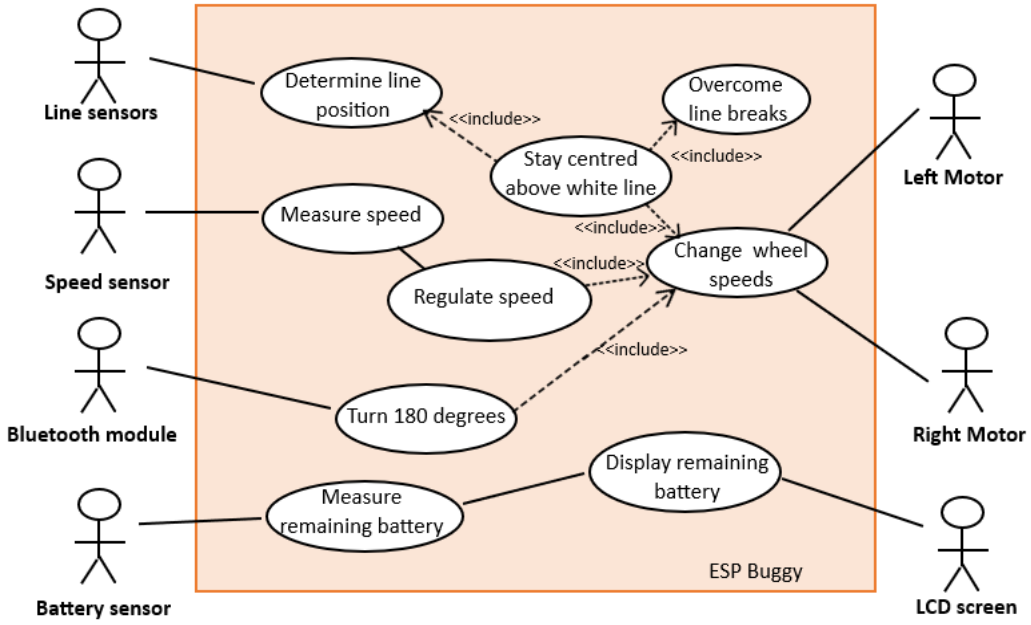


Figure 2 - Use case diagram

Determining line position
Line sensors will have varying voltages across their photo-transistor corresponding to their position over the white line, which will be passed to the microcontroller. The voltages read for line sensors in different positions will be used by the software to determine where the buggy is, and thus what adjustments are necessary to keep the buggy centred on the line.

Measuring Speed
Quadrature encoders will send pulses to the microcontroller according to the rotational speed of the axles, and this data will be processed by the software to give the buggy speed. This will be done by counting the number of pulses in a certain period, which can be used to calculate revolutions per second and velocity.

Measuring battery
The Dallas DS2781 takes periodic measurements of the battery current and passes this information to the microcontroller, which processes it and prints remaining battery percentage on the LCD screen

Turn 180 degrees
A button connected by Bluetooth will trigger an interrupt routine in the software to turn the buggy 180 degrees.

Stay above white line
Software will use proportional control to process the line position and calculate the relevant motor speed adjustments to remain on the line. The motor drive board will then pass on the required current to each motor.

Control wheel speed
Different right and left wheel speeds will cause the buggy to turn. The motors turn at speed corresponding with the voltage provided by PWM signals. A unipolar motor operation will result in less losses.

Regulate speed
There needs to be a speed limit in the software so that the buggy doesn't go too fast downhill and lose control. If the measurements from the speed sensor go above this limit, the buggy must brake.

Overcome line breaks
There may be small line breaks in the track. If no sensors are reading as above the line, the buggy should continue forwards long enough to overcome the line breaks, but still stop when the line actually ends.

2.4 Object specification - function prototypes
Here below is an example prototype for a function.

```
Class Motor: Public PwmOut{
        private:
                DigitalOut Direction, Mode;
        public:
          Motor(Pulse p, Direction d, Mode m): PwmOut(p), Direction(d), Mode(m){}

                void brake(){
                        }

                void reverse(){
                        }
        }LeftMotor(p, d, m), RightMotor(p, d, m);

float Rspeed, Lspeed;                   //0-1 variable to represent percentage of full speed
```

```
QEI QEncoder(channelA, channnelB, index, pulsesPerRev, QEI::X2_ENCODING);
int Revsps, Qstate, CurrentVelocity;
Ticker CountPeriod;

        void Qcount(){
                Revsps = QEncoder.getRevolutions();
                QState = QEncoder.getCurrentState();
                CurrentVelocity = (Revsps*wheel circumfrence)/gbr;
                QEncoder.reset();
            }
```

```
Serial hm10(D3,D2);                     //initialise bluetooth module

        void BluetoothISR(){
            }
```

```
DigitalInOut one_wire_pin(PC_12);               //battery sensor (from technical handbook)
int VoltageReading, CurrentReading;
float Voltage, Current;
```

```
class Sensor {
        private:
                DigitalIn inputSignal;
        public:
                Sensor(PinName pin): inputSignal(pin) {};

                float voltage() {
                        return inputSignal.read();
                }
}
```

3 Line Sensor Characterisation

3.1 Section introduction
A line sensor detects and identifies the position of lines. Most line sensors use optical principles. The transmitter emits light and the receiver measures the light

reflected back from the surface. When the line sensor is positioned on the target surface, the sensor measures the reflected light and detects the position of the line by analysing the difference in reflection of the light, and then generates an electrical signal that allows the microcontroller to deduce the position of the line. A suitable line sensor can identify and track lines, providing precise positioning and navigation functions for the buggy. In the experiment, the four characteristics of dark current, background illumination, variation with height, and line spread function of different line sensors were tested for the optimal one.

3.2 Comparing relevant line sensors

The experiment provided nine components for comparison, three emitters, three detectors, and a package containing detectors and emitters. TCRT5000 is included in both tables as it contains a transmitter and a sensor.

Name	Forward Current (mA)		Forward voltage (V)		Peak wavelength (nm)	Angle at half intensity (Degrees)
	typ	max	typ	max		
OVL-5521	20	30	4.0		480	± 15
TSHA4401	100	200	1.5	1.8	875	± 20
TSHG6400	100	200	1.5	1.8	850	± 22
TCRT5000	60		1.25	1.5	950	Unknown

Table 2 - Key characteristics of emitters

Name	Collector Current (mA)		Dark Current (nA)		Saturation Voltage (V)	Wavelength of Peak Sensitivity (nm)	Angle at half intensity (Degrees)
	typ.	max	typ	max			
BPW17N	50	100	1	200	0.3	825	± 12
SFH203P	80		1	10	Unknown	850	± 75
TEKT5400S	1	100	1	100	0.3	920	± 37
TCRT5000	1	2.1	10	200	0.4	950	Unknown

Table 3 - Key characteristics of detectors

3.3 Pairing emitter and detector

The following emitters and detectors were paired due to their similar peak wavelength.
Emitter: TSHG6400 - 850 nm, OVL5521 - 480 nm
Detector: SFH203P - 850 nm, BPW17N - 825 nm, 5 MΩ LDR - visible light

3.4 Data analysis

3.4.1 Dark current

Dark current is the small current flowing through the sensor without detecting any light. Turning the LED off and blocking any light from entering the phototransistor, the voltage across the resistor is 0.02 mV. Therefore the dark current is 0.02 V /1600 =

0.0125 μA = 12.5 nA, which is close to the value in the TCRT5000 table data. Having a low dark current with the sensor is beneficial as dark current can be considered like noise for the sensor. The smaller this value is, the more accurate the data. 12.5 nA is relatively a small value, therefore the detection error caused by the TCRT5000's own interference is small.

3.4.2 Background illumination

With the TCRT5000 pointed at a bright light that is approximately 1 m away, the measured values are as follows:
The largest voltage across the resistor is 0.0215 V,
The largest current is 0.0215 V / 1600 = 0.0134 mA
Compared with the current value of the circuit when the line sensor is located on the black and white lines, it is very small. Therefore, background illumination has little impact on the TCRT5000.

3.4.3 Variation with height

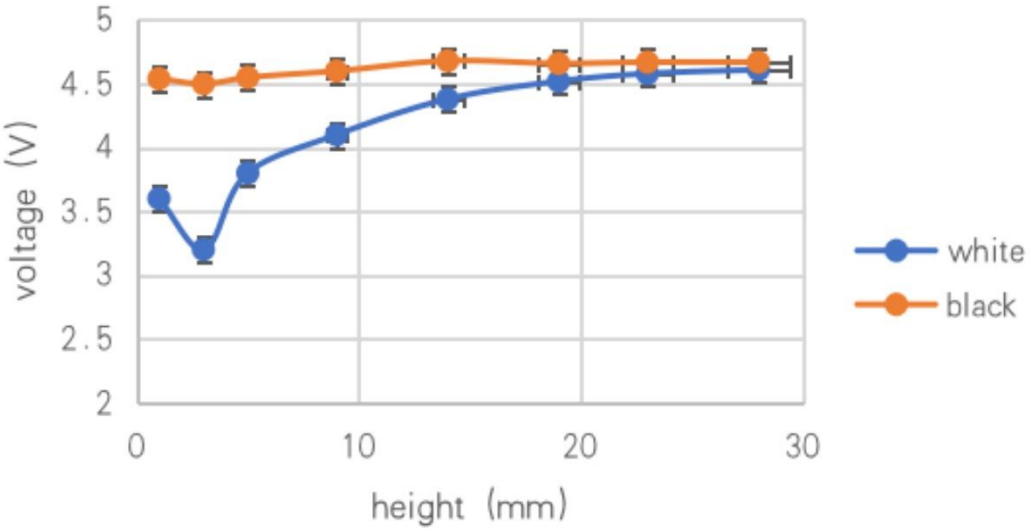


Figure 3 - Vertical Height Above the Black and White Sections of the Track vs Voltage for the TCRT5000

Figure 3 shows the different voltage outputs of the sensor above the black track and above the white line at different heights. It is obvious that the voltage difference is the largest when the height is 2-3 mm, indicating that the sensor can better distinguish the black and white areas at this height, which will help the buggy accurately locate and move along the white line. When approaching and moving away from the most effective height given by the datasheet of TCRT5000 (which is 2.1 mm [1]), the sensor output voltage difference becomes smaller, and sensor's ability to distinguish black and white areas decreases.

For other sensors:

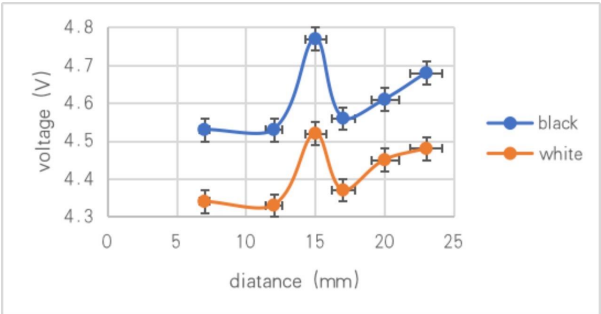


Figure 4 - TSHG6400 & BPW17N

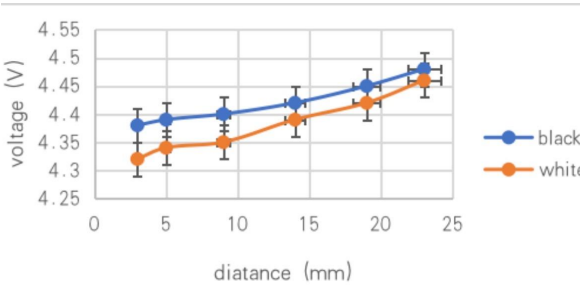


Figure 5 - OVL5521 & 5 MΩ LDR

The output voltage difference between the black area and the white area at different heights is stable and small, indicating that the two sensors have poor recognition capabilities for white lines.

3.4.4 Line spread function

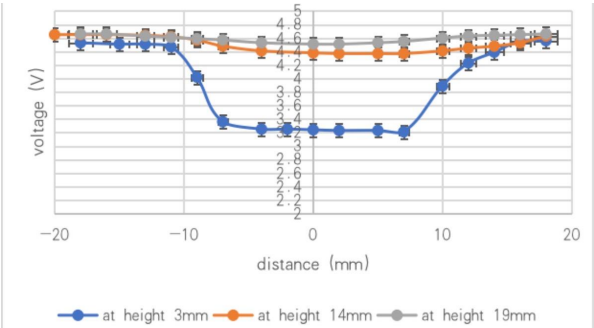


Figure 6 - Voltage vs. horizontal distance from white line for the TCRT5000

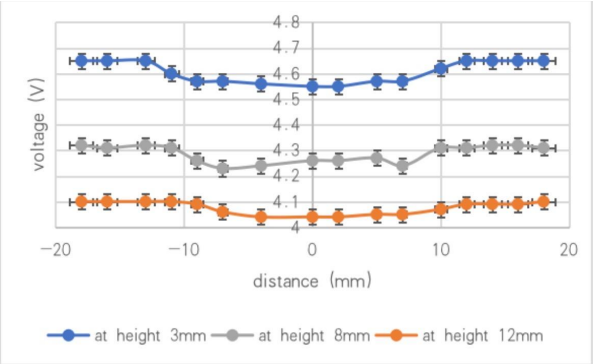


Figure 7 - Measuring voltage across phototransistor for OVL5521 & 5 MΩ LDR as distance from centre of white line changes

In this experiment, the sensor position changes from being over the black track area to being over the white line and then on to the black area again. The voltage output data of the sensor was recorded as it moved along the track at different heights. Figure 6 shows that at a height of 14 mm and 19 mm, the voltage output is close to being a flat, horizontal line. The sensor can hardly distinguish between the two surface types, as seen by the voltage output difference across the transistor being almost equal for the track and white line. At a height of 3 mm, the voltage output above the white line is noticeably smaller than the voltage output above the black area, and the voltage output value at the edge of the white line changes significantly, which shows that the sensor recognizes the edge of the line well. Overall, results suggest that the sensor responds the best when it is close to the track, at around 3 mm above the surface.

For OVL5521 & 5 MΩ LDR, since the spectral bandwidth of OVL5521 is 400-800 nm, the light emitted by the emitter is visible light, so outside light at different heights will affect the work of the detector, resulting in the vague identification of the white line for this sensor.

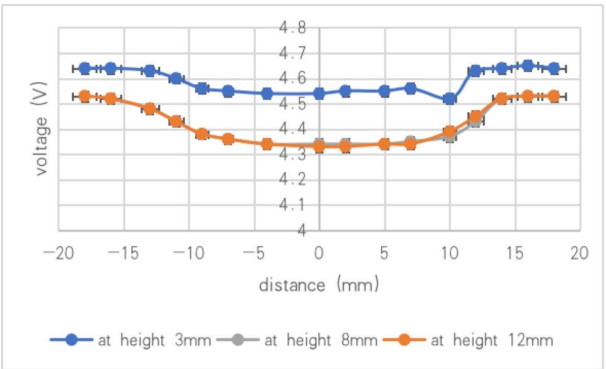


Figure 8 - TSHG6400 & BPW17N

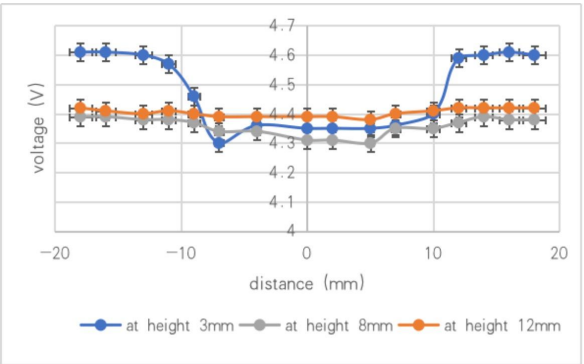


Figure 9 - SFH203P & TSHG6400

For TSHG6400 & BPW17N and SFH203P & TSHG6400, the difference in voltage output when located above the black area and the white line is small. Even if there is a certain difference, it is far from enough to clearly distinguish the white line. This shows that this sensor is not suitable for locating the buggy's trajectory.

3.4.5 Errors

An error bar is added to each chart, and an error range of 5% is set for the x-axis value. Since the y-axis value itself is large and the fluctuation is small, using an error range of 5-10% will produce too large an error. Therefore a fixed error value of 0.03 is set for the y-axis value.

Potential errors when conducting the experiment included external interference due to bright lights in the laboratory.
More specifically, in Figure 4, the results for the TSHG6400 & BPW17N have a sudden peak at 15 mm, this is an anomaly caused by unexpercted lighting changes.

3.5 Line sensor selection

Considering key factors provided by experimental data such as dark current, background illumination, variation with height, and line spread function, TCRT5000 will be the final line sensor used in the buggy. The reasons are:

1. It has a small dark current, which reduces measurement interference.
2. According to the background illumination section, compared with the current when measuring the white line, the interference current generated by strong light is very small, so the TCRT5000 can work under direct sunlight.
3. The optimal working height of TCRT5000 is about 2.1mm, which is similar to the distance from the buggy base to the track. When the sensor is mounted on the buggy, it can work with peak sensitivity.
4. It has the highest recognition of white lines and the strongest positioning ability.
5. It comes as one complete package, reducing errors caused by manual soldering of components.

In order to cope with line-breaks problem, a ‘delay’ will be set to the buggy. More precisely, there is a time lag between when the sensor detects the white line and when the buggy moves. In this way, when the buggy is above the broken white line, it will still move forward according to the instruction of the sensor detecting the white line at the last moment without losing its direction.

4 Circuit Diagram of Line Sensors

4.1 Introduction

This section contains the schematic diagram and the PCB layout diagram for the full line sensor circuit. These include all five TCRT5000 line sensors, resistors, and pin interfaces for connecting to the microcontroller. The wiring diagram shows the connections needed between the PCB and the microcontroller. Also included is a stripboard diagram for a single line sensor, which was used in the lab for testing sensor characteristics. The software provided with for the stripboard was not advanced and had no 8 pin DIL socket component, so it has been represented using a different component.

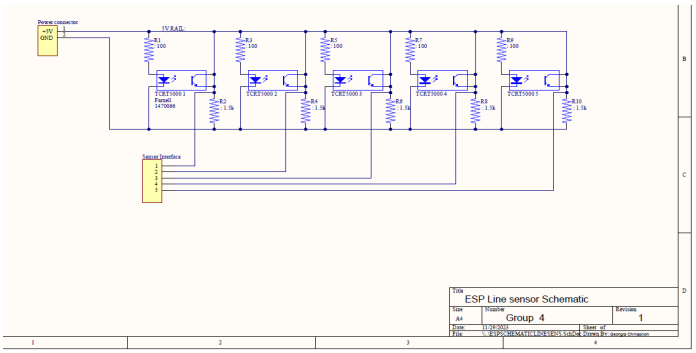


Figure 10 - Schematic diagram of the circuit

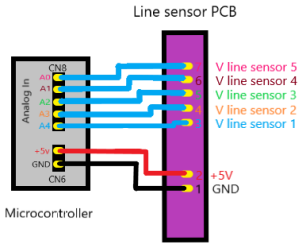


Figure 11 - Wiring diagram

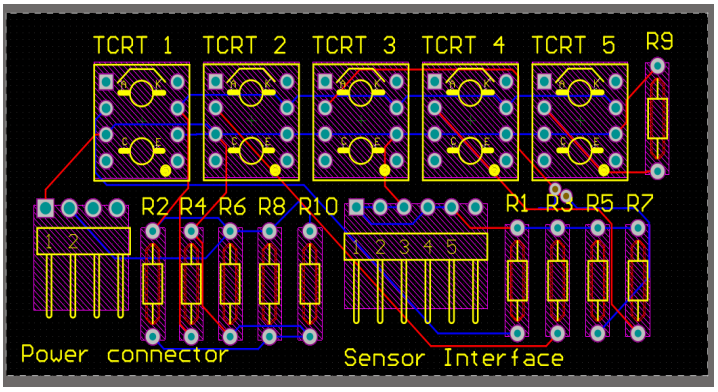


Figure 12 - PCB design

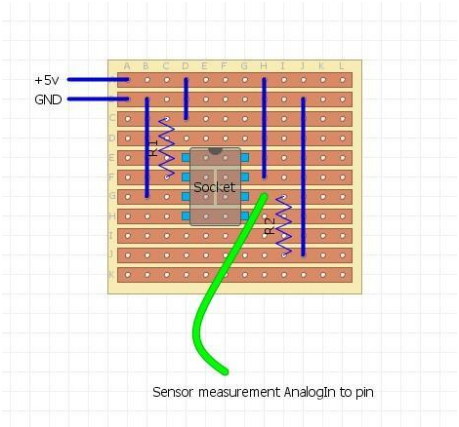


Figure 13 - Stripboard diagram used for line sensor lab

5 Non-Line Sensors

5.1 Encoders

Encoders are sensors used to convert mechanical motion into electrical signals. In ESP programs, encoders are vital for speed measurement, direction detection, position control, and feedback. The microcontroller receives feedback by counting pulses generated by the encoder for each rotation angle. Additionally, the speed of rotation of the encoder can be determined by counting the number of pulses over a certain period of time. If used, quadrature encoders enable the microcontroller to determine the direction of rotation, i.e. clockwise or anti-clockwise, by comparing the pulse signal sequences from two channels.

5.2 Battery monitor

Battery monitors, typically implemented with integrated circuits, employ the Dallas DS2781 to measure battery voltage and cumulative current. The DS2781 utilises continuous voltage and current measurements to calculate battery power consumption. If the battery voltage remains relatively constant during the test, the energy consumed by the battery can be approximated using the formula:

$$E=\int v(t)i(t)dt \approx V_{batt} \sum_k i_k \Delta T$$

i_k is the samples at each interval, ΔT is the interval, V_{batt} is the battery voltage, and E is the energy use.

The microcontroller tracks battery energy usage by interpreting digital voltage and current signals from the DS2781. Using the mBed code library and additional programming, these digital signals are converted into actual voltage and current values. These values can then be processed to determine the battery percentage remaining.

5.3 Motor current

Motor current is measured by adding a 0.1Ω resistor to the circuit, which produces a small voltage difference across the resistor as current passes through the motor. This voltage difference is proportional to the motor current and can therefore indicate the magnitude of the current. The sensor converts this voltage difference into a voltage signal, amplifies it through an isolation amplifier, and sends the signal to the microcontroller. The microcontroller uses programming via the mBed code library to convert the digital voltage signal into an actual current value. This enables the microcontroller to monitor the motor current in real time.

5.4 Implementation

To implement the sensors mentioned for our buggy, they must be connected to the microcontroller - most can be connected by any GPIO pin, but the encoders can connect to the built-in Quadrature Encoder peripherals (TIM2 and TIM5). The encoders must be connected to the wheels of the buggy to detect mechanical motion. The battery monitoring sensor tracks battery power consumption by connecting to the battery circuit and reading voltage and current data. The current monitoring sensor needs to be added to the motor circuit to monitor the status of the motor by measuring the motor current. Code is then written to convert the digital signals the microcontroller receives from the sensors into actual voltage, current or motion parameter values.

5.5 Alternative sensors

Absolute encoder

Pros:
-Unique position information: Each position has a unique encoding, which enables immediate repositioning to the exact position again, even after a power failure and then restart.
-High accuracy and resolution: Absolute encoders provide highly accurate position information and are suitable for applications where accuracy is critical.
-Suitable for multi-axis control systems: Absolute encoders provide an efficient solution for systems that require synchronisation and precise control of multiple axes of motion.
-Eliminates the need for zeroing operations: reduces start-up time and potential positioning errors, making it particularly suitable for environments with frequent power failures.
Cons:
-Typically more expensive compared to four-phase encoders.
-More advanced data processing capabilities are required.
Interfaces:
Digital interfaces, e.g. serial or parallel, and industrial communication standards.

Hall Effect Sensors

Pros:
-Non-Contact Measurement: Hall effect sensors are non-contact sensors, eliminating the need to physically touch the current conductor or circuit under test. This reduces the risk and complexity of the measurement.
-High accuracy: Hall effect sensors are typically highly accurate, providing accurate current measurements.
-Fast Response Time: Hall effect sensors have a very fast response time and can monitor changes in current in real time.
Cons:
-Expensive, especially in small numbers for a standalone project like this.

6 Control System

6.1 Detecting the line

For the buggy to successfully navigate the track, the buggy uses line sensors to detect the white line on the track. An array of sensors will provide a voltage output depending on the reflectance of the surface beneath them. According to the line sensor characterisation, the sensor detection of a white line corresponds to lower voltage. Each sensor will be constantly getting new inputs due to change in surfaces and the output will be sampled and processed to distinguish the position of the line in relation to all the sensors and thus detect any changes of the direction of the track.

6.2 Controlling the direction & speed of buggy on the line

As the buggy uses two DC motors with no physical brakes or steering ability, the buggy's direction and speed are determined by the motors, therefore controlling the motors' rotation direction and speed will control the direction and speed of the buggy.

H-Bridge Controlling Motor Direction:

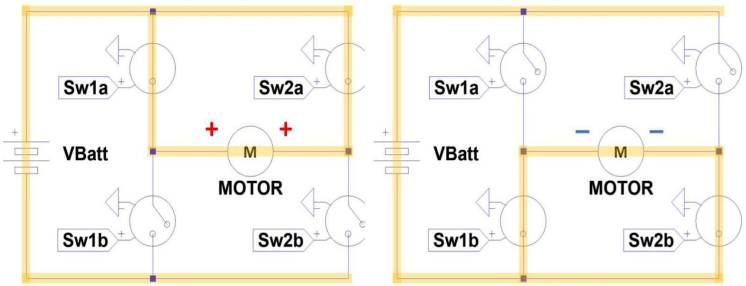
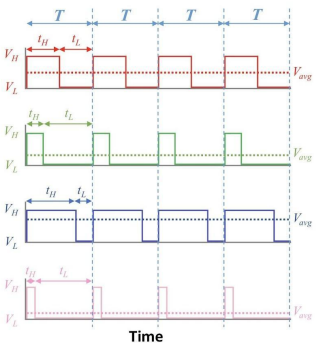


Figure 14 - H-bridge control [2]

An H-bridge is a critical circuit configuration employed to precisely control the direction of a motor. Comprising four switches, the H-bridge enables the alteration of current flow within the motor winding by strategically opening or closing these switches. Specifically, when two diagonally positioned switches are simultaneously closed, current flows through the motor, inducing rotation. By altering the switch states, the current direction is reversed, thereby reversing the motor's rotation direction. +Vbatt will output the full speed in either forward or reverse direction. Consequently, the design flexibility of the H-bridge allows for nuanced adjustments in current flow within the motor, facilitating accurate control over the motor's rotational direction.

PWM Controlling Motor Speed:



$$T = t_H + t_L$$

Average Voltage:

$$V_{avg} = \frac{1}{T}\left(t_H V_H + t_L V_L\right)$$

Duty cycle:

$$D = \frac{t_H}{T}$$

Figure 15 - PWM waveform structure [2]

The speed of each of the buggy's motors will be controlled by a PWM signal generated by the microcontroller and passed to the motor drive board.

Pulse Width Modulation (PWM) is an effective technique for controlling motor speed. By periodically adjusting the switch states of the power source in short intervals, PWM modulates the average voltage applied to the motor. This modulation is achieved by generating a pulsating signal with alternating high and low voltage levels. In the PWM signal, the ratio of the duration of the high voltage level to the overall period is known as the duty cycle, and altering this directly impacts the motor's average voltage. Increasing the duty cycle, or the proportion of time the signal is in a high state, elevates the motor's average voltage, thereby increasing its speed. Reducing the duty cycle lowers the average voltage and decreases the motor speed.

To give a specific example of buggy application, if the buggy shifts to the right of the white line, the line sensor on the right faces the black area and outputs a high voltage, exceeding the set voltage threshold. Therefore, the ratio of the duration of the high voltage level is higher, that is, the motor speed on the right side is higher. Meanwhile, the line sensor on the left side faces the white line and outputs a low voltage, which is similar to or lower than the set voltage threshold, so the ratio of the duration of the high voltage level is lower, and the speed of the left motor is lower. The right wheel is fast and the left wheel is slow, causing the buggy to drive left and return to the white line track. Consequently, PWM technology allows for the continuous and controlled adjustment of motor speed, providing high flexibility for the buggy's speed.

In addition, a P controller can be added to monitor the overall speed of the buggy. The P controller will compare the buggy's current driving speed with the set speed. This means that when the buggy slows down on the uphill slope or speeds up on the downhill slope, the controller will control the speed to be constant to stop the buggy from stalling or travelling too fast.

6.3 PID controllers and bang-bang controllers

6.3.1 PID controllers

The Proportional-Integral-Derivative controller, or PID contoller, is a widely used feedback controller in industrial and automatic control systems. It consists of three fundamental components:

1. Proportional (P): Corrects based on the magnitude of the error. Specifically, the output is the error multiplied by a constant known as the proportional gain.
   Pros: Helps reduce steady-state error, responds quickly to changes.
   Cons: May lead to overshooting and oscillations, doesn't eliminate steady-state error entirely.
2. Integral (I): Corrects based on the cumulative error over time, the output is directly proportional to the integration of the error.
   Pros: Eliminates long-term steady-state errors, ensuring the system aligns with the setpoint in a stable manner.
   Cons: Introduces the potential for integral windup, which can cause overshooting.
3. Derivative (D): Corrects based on the rate of change of the error. It calculates the derivative of the error, multiplies it by the derivative gain, and adds this result to the output.
   Pros: Improves transient response, suppresses rapid changes, reduces

overshooting, and helps stabilize the system.
Cons: Sensitive to noise in the signal and always amplify high-frequency components.

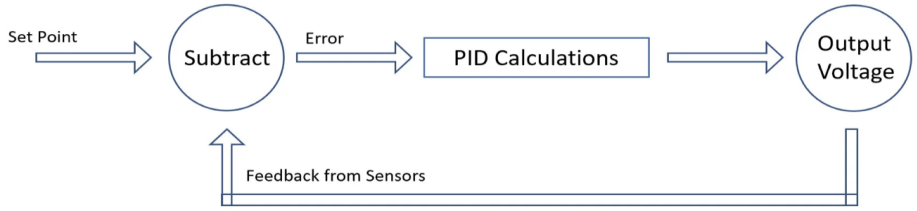


Figure 16 - PID controller feedback system

For the PID controller:
Pros: It is applicable to a variety of systems, providing good performance in many scenarios. It also adjusts in real-time based on system feedback, enabling quick and effective responses to system changes. The three components of PID controllers (Proportional, Integral, Derivative) can be individually adjusted, providing strong flexibility to meet the requirements of different systems and reducing steady-state error, maintaining the system close to the setpoint over extended periods.
Cons: Parameter tuning may require experience or debugging and incorrect parameter configurations may lead to instability or excessive oscillations. PID controllers may perform poorly in some nonlinear or time-varying systems, requiring more advanced control strategies.

In summary, the PID controller is a powerful and flexible control tool, but careful parameter tuning is necessary to meet the specific requirements of the application.

6.3.2 Proportional vs bang-bang controllers

A bang-bang controller is a type of control system that switches between full-on and full-off states when the system reaches or exceeds a setpoint. It is a discrete process and uses binary logic, either fully opening or fully closing, to maintain the system near the setpoint. This control method is straightforward but due to the width of the hysteresis gap and the inertia in the control process, there will be an oscillating error signal, usually in a sawtooth shape, around the desired setpoint value.

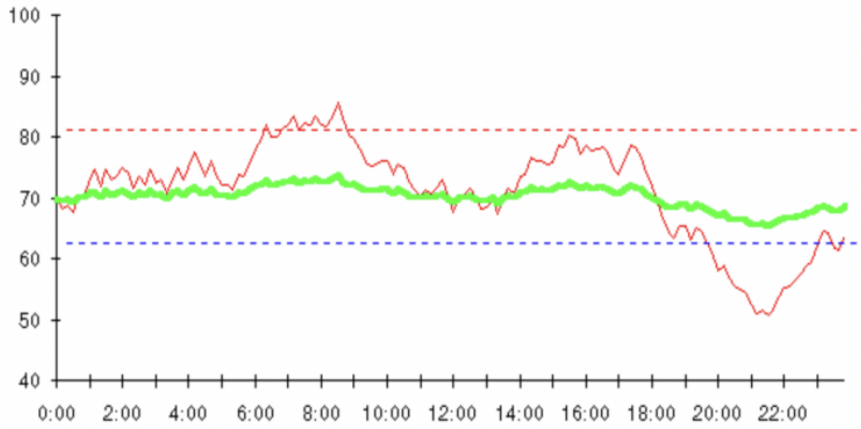


Figure 17 - typical output figure of Bang-bang vs proportional controller [3]

In Figure 17, the red line is the bang-bang controller output and the green line is the Proportional controller output and the x-axis is time. According to the figure, there is a signature of bang-bang controller: nothing happens until one of the limits is hit, the output varies freely between them. When the system exceeds a limit, it's hauled back

within range, then allowed to drift again.In comparison, the Proportional controller offers a stable and smooth value, which can make systems run smoother than bang-bang. As with negative and positive feedback, proportional control lends stability to a system, while bang-bang control tends to oscillate between the extremes.

In summary, a proportional controller provides a more nuanced and continuous response to errors but comes with the challenge of potential oscillations. On the other hand, a bang-bang controller is straightforward and robust but tends to cause large oscillations and might lack precision in certain contexts.

6.4 Control objects and applicable control algorithms

The main concern for the buggy is to control its speed and direction to ensure that the vehicle moves accurately along a path. This requires the control algorithm to be able to regulate the speed of both wheels to maintain a straight line, or to steer and turn around by adjusting the speed difference between the two wheels. In this case, proportional control (P-control) is considered the most suitable algorithm for the task.

6.5 Proportional Control

Proportional control (P control) is a basic form of feedback control that works by making the control output proportional to the error (i.e. the difference between the desired and actual values). In this algorithm, the error signal is multiplied by a proportionality factor (P-factor) to produce the control signal. This method adjusts to the deviation of the trolley from its intended path and is able to respond quickly to changes in the state of the system.

In this project, the main reason why proportional control has an advantage over other algorithms (such as PI, PD or PID control) is due to its simplicity and intuitive feedback mechanism, while it also ensures an appropriate control response. This control algorithm reduces the complexity of system design and debugging and is particularly suitable for this project. Another benefit of proportional control is the flexibility to adjust the strength of the response to deviations by adjusting the P-factor, thus adapting the control effect to the actual situation. This flexibility makes proportional control ideal for achieving the project goal of navigating around a track.

6.6 Interrelationship between control algorithms and sensor design

The choice of different control algorithms can have a significant impact on how the control system works, which is closely related to the performance of the sensor. P control requires fast and accurate data feedback from the sensor to help the system correct errors in a timely manner. Proportional-Integral (PI) control, on the other hand, focuses more on a constant and stable supply of data from the sensor to help the system remain accurate over time. Proportional-differential (PD) control, on the other hand, relies on the sensors to quickly capture changes and predict what might happen next. More sophisticated Proportional-Integral-Differential (PID) control combines all of these requirements and requires sensors that excel in accuracy, continuity, and responsiveness. The sensors must be able to meet the specific needs of the chosen control algorithm to ensure that the entire control system operates accurately and efficiently.

In this project, the successful operation of the P-control algorithm relies heavily on the design and performance of the sensors. Proportional control is the process of adjusting the speed and direction of the buggy based on the information measured by the sensors. Specifically, when the buggy deviates from the white line, the proportional controller adjusts the speed difference of the motors according to the

degree of deviation measured by the sensors, thus returning the buggy to its intended path. The core of this proportional control strategy is that the control action is proportional to the error, meaning that the larger the deviation, the stronger the response of the controller. The effectiveness of the control algorithm is therefore very dependent on the quality of the data provided by the sensors. If the sensor is able to accurately measure the deviation of the trolley with respect to a predetermined path, this not only significantly improves the adaptability and responsiveness of the control algorithm, but also increases the accuracy and reliability of the overall control system. In this case, the proportional control effectively corrects the deviation of the buggy from the path and reduces the degree of swaying and deviation of the cart from the path through regulation, resulting in a smoother and more accurate travelling path for the buggy.

6.7 Impact of relationship between algorithms and control objects on the sensor setup, circuit design, and interface with the microcontroller

In a project using proportional control algorithms, there are several important requirements for the sensor setup. Firstly, the sensors need to be very accurate and be able to tell the control system exactly how far the buggy is from the intended path. Second, the sensors must be responsive and able to detect changes in the vehicle's position immediately so that the control system can react quickly. Further, the sensors should be stable and reliable, continuously providing accurate information so that the control system can continuously make the correct adjustments. Finally, the connection between the sensor and the microcontroller should process the data quickly and accurately to ensure that the information passed to the control system is correct.

When using proportional control algorithms in a project, the circuit design needs some special adjustments to support this type of control. It must be able to process data from the sensor quickly and accurately, translating the sensor's signals into instructions that the microcontroller understands. Then, if the signals from the sensor are too weak, the circuit may need to add an amplifier to allow the microcontroller to read them more clearly. In addition, the entire circuit has to be designed to ensure a quick response so that the buggy can instantly adjust its direction of travel or speed. It is also important to make sure that the circuit is stable and reliable, just as it is important to make sure that the buggy drives as planned in all road conditions.

The interface to the microcontroller is subject to numerous special influences. The microcontroller needs to be able to quickly read and process the information from the sensors, calculating in real time the gap between the actual position of the buggy and the intended path. The microcontroller then has to send commands based on these calculations, such as adjusting the speed of the wheels, to ensure that the buggy is travelling as planned. If the sensors are sending analogue signals, the microcontroller also needs to be able to convert these analogue signals into digital signals for calculations. In addition, if the system allows the user to manually adjust settings, the microcontroller's interface must be able to accept these user inputs.

6.8 Sensor implementation

The sensors will be implemented in a straight line of 5. This will be sufficient to cope with responding to bends in the track whilst maintaining as simplistic of a design as possible, which will hopefully reduce any difficulties during manufacturing. The choice to have an odd number of sensors was decided as an even amount would cause the buggy to try to keep the line at the exact position between the two central sensors to maintain equal voltage levels. Placing one sensor right at the centre of

the buggy means it will only reposition if voltage at either side of this sensor increases too much, rather than adjusting unnecessarily when it is only slightly off the line. This should also decrease chances of over-adjusting.

It was decided that five sensors was the optimal amount. Three would not give enough information, as it would be more difficult to implement to what degree the buggy should turn if there is only one sensor on either side, and seven would be a tight fit on the buggy, and more program lines are required to account for additional situations.

6.9 Control algorithm implementation

To implement the control algorithm, the software should return values in quick intervals based on where the line is. For example, it may return a large value if one of the outer sensors detect the most light, and small value if the central sensor detects the line. The software then sends signals to the motors to increase or decrease their speed according to this value, with a single value to tell the buggy which direction it must rotate. Following the above example, the left motor increases speed and right motor decreases speed in proportion to the magnitude of the value if the signal indicates the line is to the left, and the reverse would apply if it indicates that the line is to the right.

6.10 Line breaks and sunlight

To prevent line breaks causing issues for the buggy, the system will be programmed to move for a little time after it stops sensing the line. Therefore, if it is at the end of the track, it will stop moving soon after the track ends, but if there is only a break in the track, it will continue in a straight path until it senses the line after the break, and regular pathing recommences. For sunlight, simply adding a shield above the sensors to block enough light to prevent interference.

7 Hardware

7.1 Introduction

This section covers the design of a suitable chassis that is necessary for all the parts to be mounted to. The chassis faced numerous size constraints as a result of restrictions from the track. Numerous different designs were sketched, and then of those designs three were then modelled in full in SolidWorks. Of these three the best design was chosen, the design chosen was the smallest of all the designs which is a great advantage as it makes it easier to avoid hitting the walls on the track.

7.2 Selection of best design

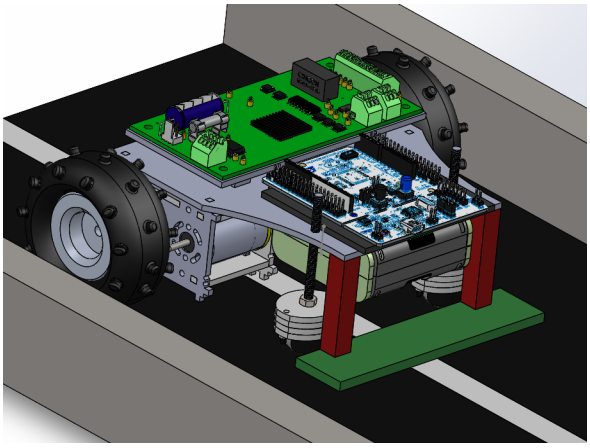


Figure 18 - Design 1

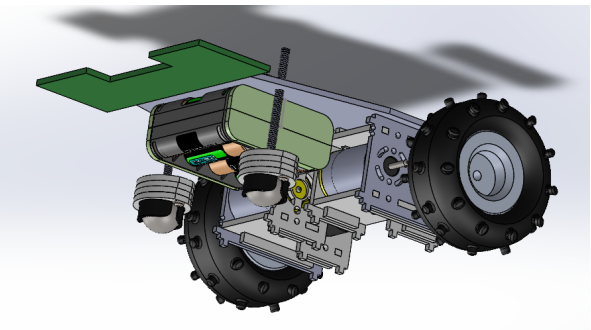


Figure 19 - Design 2

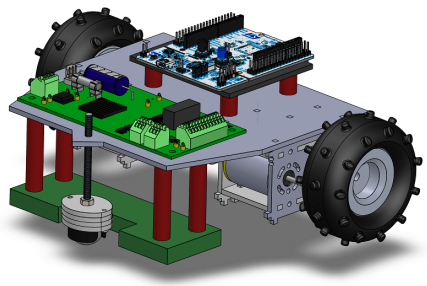


Figure 20 - Design 3

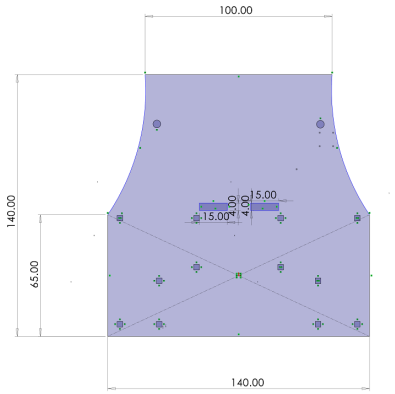


Figure 21 - 2D Dimensioned design of chassis in Design 1

Each of the three designs above all had different benefits and disadvantages, however design 1 is the optimal one. Design 1 takes beneficial characteristics from the other design, for example: having the batteries underneath the chassis for a low centre of gravity, increasing stability; having two castor balls, which distributes the load over more points of the chassis and thus allows for a greater load on the chassis. Design 1 also benefited from being the smallest of the three designs. A smaller buggy will make it much easier to navigate the course without hitting the walls, especially during the 180 degree spin. Figure below shows the precise dimensions of the chassis. It is effectively a square shape(each side 140mm) however the left and right side have curved sections to remove weight and make the front of the buggy narrower, which means that during 180 degree rotation the chassis is less likely to collide with the walls.

7.3 Stress analysis of chassis

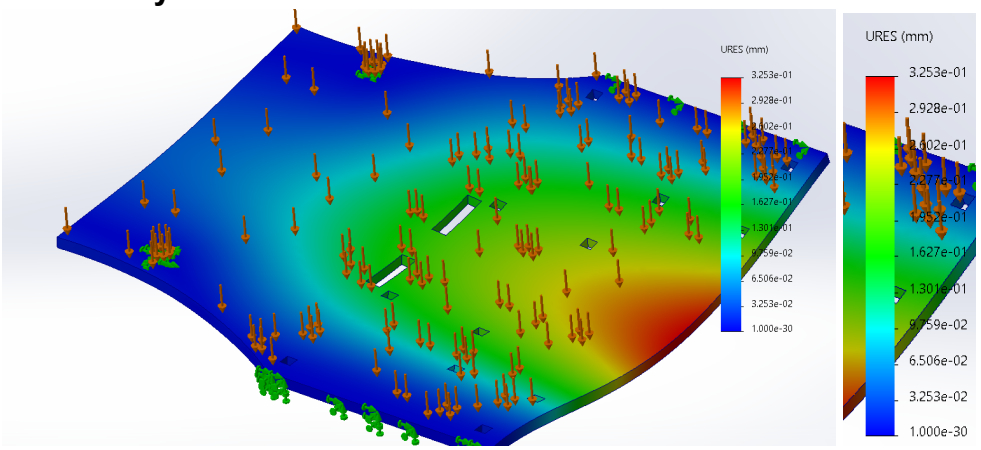


Figure 22 - Deflection values of the chassis

As seen in Figure 22, using solidworks a stress analysis simulation was run on the chassis and the results were very encouraging as the deflection was minimal. Firstly, as part of the simulation the force applied to the whole chassis was 23.4 N, this value was used as it is double the force as a result of the weight of all the standard components and weight of the chassis. Additionally, the green arrows in the figure show the fixed points, these are points in which the chassis will receive a support force from either the castors or the gearbox assembly.The fact that the chassis can withstand double the force expected on it is excellent. Additionally, the use of two castor wheels has allowed the force to be distributed more evenly across the chassis than if one was used, resulting in less potential deformation. Deformation is understandably bad for the chassis as it could lead to the whole buggy breaking.

7.4 Dimensioned 2D drawings and design considerations

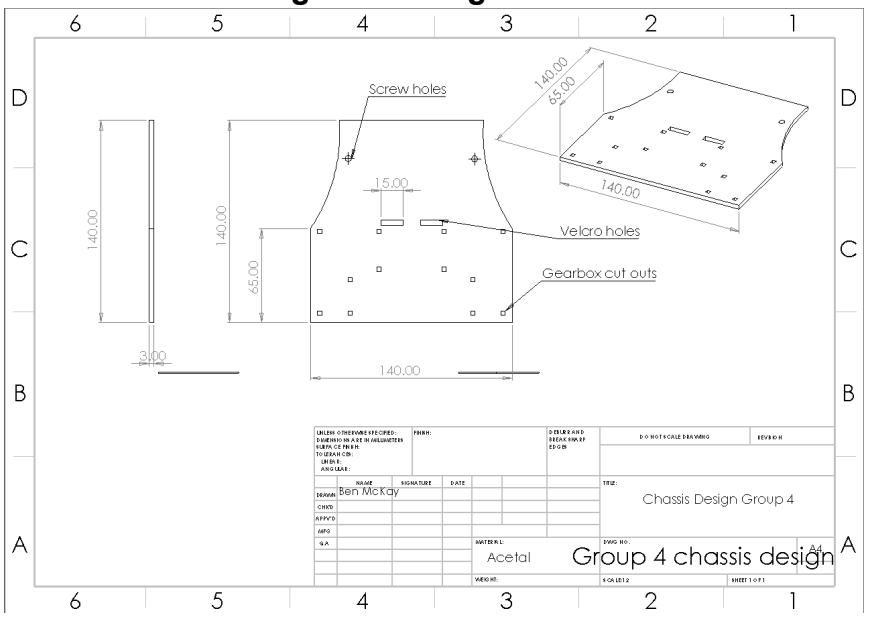


Figure 23 - Dimensioned 2D drawings

As shown in Figure 23, the dimensions in 2D show the chassis is at its widest point of 140 mm, and narrowest at 100 mm. This is a result of the chassis narrowing towards the front, which has the benefit of reducing the likelihood of collision with the side walls as the buggy rotates on its rear axle and the front swings towards the wall. The length at 140 mm, combined with the narrow width, should mean that the track is 140 mm either side of the white line, so the track is 297 mm wide in total. Also on the above diagram is the thickness of the material, which is 3 mm. This is thick enough to ensure structural integrity whilst minimising weight.

A major design decision was the placement of the gearbox (and thus the axle) as this decided the turning circle and point of rotation of the buggy. By moving the gearbox closer to the centre, the turning circle's radius is reduced, which is very beneficial. However, it moved too far forward and there is no space for the other components, as well as making the buggy prone to tipping backwards. The gearbox is attached to the chassis using a Mortise and Tenon joint, where small pieces of the gearbox stick out and fit through cut sections in the chassis. This allows the gearbox to be seamlessly inserted into the chassis, although a glue or another adhesive would be needed to permanently secure the gearbox. The gearbox placement is such that the rear axle, in which the wheels are mounted, are 35 mm from the very rear of the chassis. This distance allows a long wheelbase which ensures stability and a balanced weight distribution, resulting in improved traction for the wheels. However, the gearbox is still placed 35 mm from the rear so that the wheelbase is not too long. By shortening the wheelbase, the buggy becomes more manoeuvrable, which aids the buggy in navigating the track.

7.5 Material selection

Below, in Table 4, is an analysis of common materials and their properties. To select the optimal material it is imperative to consider the use case and requirements of the buggy. For example, carbon fibre has much greater tensile strength than all of the other materials, however, the buggy isn't going to face such a large amount of tensile stress, and it is very expensive (16.7 times more expensive than mild steel). What the chassis requires is a material that is cheap, lightweight, easy to cut, and strong enough to support a fairly limited force from the weight of components. Therefore, acetal is the best choice as it provides the structural integrity required, whilst being

the second cheapest material and by far the easiest to manufacture the chassis with. Acetal can be laser cut from sheets, or used in 3D printers. In contrast, aluminium and steel require much more time and power to be cut from the laser cutter, and are both heavier than acetal.

Material	Acetal	Glass-Reinforced Laminate	Aluminium	Mild steel	Carbon Fibre
Density (Mg/m^3)	1.8	1.9	2.7	7.8	2
Flexural strength (MPa)	91	255	310	414	320
Ultimate Tensile Stress (MPa)	67	175	310	414	4000
Young's Modulus (GPa)	2.8	11.5	69	207	500
Cost (£/kg)	1.50 [4]	3.10 [5]	1.71 [6]	1.01 [7]	16.93 [8]

Table 4 - Comparison of different chassis material choices [9]

8 Conclusion

The aim of this report is to understand and determine various technical aspects of our buggy's design that will allow it to perform its function of following a line around a track in the most efficient and error-free manner. This included choosing our sensors and their arrangements, exploring how to implement the required functionality of the buggy in our software, designing the chassis, and choosing the algorithm we will use to control the buggy's movement.

Assumptions:

- Buggy max speed is 1.71 m/s, from calculations in DR1
- Track width (297 mm) and max line break size (6 mm)
- Microcontroller has sufficient storage (512 kB) and pins (81 I/O ports) to interface with all sensors and run the program without difficulty

The use case and context diagrams give a good perspective on the overall functionality of the buggy and how peripherals, software, and hardware will interact. The microcontroller is central to this project, as it interfaces with all of the peripherals and runs the software that processes the signals received. Some functions are less important than others, such as battery monitoring, and can be removed if we end up facing restrictions in the microcontroller's software or interfacing capability. Other sensors and functions, however, are essential to the buggy's primary function of following the track: the quadrature encoders will supply feedback about the buggy's speed and direction, which is vital in our feedback-based control system. The non-line sensor section of the report explores absolute encoders as a possible alternative to these, but their advantages of better accuracy and immediate position reading after startup would not increase the buggy's performance enough to justify their complexity and price.

The chassis design was chosen with consideration to optimise performance, with 3 mm plates for lighter mass, a narrower body than other iterations to reduce the chance of collision with track walls when turning, and the battery under the chassis for a low centre of gravity. The rear axle is placed 35 mm from the back of the chassis, balancing a long wheelbase to ensure stability with the manoeuvrability allowed by the gearbox being closer to the centre. The material chosen was acetal, which is cheap and easily manufactured while still having the structural integrity needed. This design will ensure our buggy is stable, well balanced, and can navigate without issue.

Through line sensor characterisation, we determined that the TCRT5000 was the best sensor to use due to the wide voltage gap in its response to different track colours (4.5 V for black and 3.2 V for white at its optimal operating distance) and it is already paired, reducing room for error when manufacturing the buggy. It also has a peak detection wavelength of 950nm, which is less likely to be interfered with by sunlight, as sunlight is composed mainly of lower wavelength light.

We have chosen to use 5 of these sensors with proportional control to maintain the buggy's position over the line. This control system is simple to implement and gives a smooth and regulated output, as opposed to a different method such as bang bang control, which tends to deal with only extremes. Proportional control requires fast and accurate data sampling to correct errors in the vehicle's trajectory fast enough that the necessary adjustments can be made by the microcontroller. The sampling rate of the sensors will be 400 Hz so that the buggy can have a fast response to track changes, and thus follow the white line with accuracy.

We have chosen to use 5 sensors because an odd number of sensors means there will be a central sensor to remain above the line, which will require less frequent correction than two central sensors whose voltages must be kept equal. 5 line sensors is enough to provide the information the buggy needs, without necessitating a very wide PCB that could obstruct turning and extra peripheral interfacing with the microcontroller. Having as few as possible will optimise performance, especially when the sensors require a high sampling rate.

9 References

[1] "Reflective Optical Sensor with Transistor Output", Vishay Semiconductors, (ver. Jan. 1 2022), https://en.maritex.com.pl/product/attachment/152494/028af91734a44712540ec7d379046d75 (Accessed Nov. 30 2023)
[2] L. Marsh, Week 8 PWM/Control. [Online]
[3] J. Walker, "Bang-bang vs. proportional control", https://www.fourmilab.ch/hackdiet/www/subsection1_2_3_0_5.html (Accessed Nov. 30 2023)
[4] Acetal cost https://www.professionalplastics.com/ACETAL_RESIN (Accessed Nov. 29 2023)
[5] Glass-Reinforced Laminate cost https://gfpengineering.co.uk/grp-fabricated-products/grp-sheet.asp (Accessed Nov. 29 2023)
[6] Aluminium cost, https://www.lme.com/en/Metals/Non-ferrous/LME-Aluminium#Summary (accessed Nov. 30 2023)
[7] Mild steel cost https://mepsinternational.com/gb/en/products/world-steel-prices?#product-unit-measurement-conversion-form (accessed Nov. 30 2023)
[8] Carbon fibre cost https://www.procurementresource.com/resource-center/carbon-fibre-price-trends (accessed Nov. 30 2023)
[9] L. Marsh, "ESP Handbook Technical", 2023 (Accessed Nov. 30 2023)